



Photo by Erik Karits on unsplash

Lecture 10

—

XGBoost

Trauermantel (*Nymphalis antiopa*)

- ◆ Sein Name klingt nicht gerade fröhlich, ist wohl jedoch seiner samtig-braunen Flügeloberfläche geschuldet. Diese wird gerahmt von einer Borte blauer Punkte und schließlich einem cremefarbenen, gezackten Rand. Nach der Überwinterung verblasst der gelbe Rand in weiß.
- ◆ Der Trauermantel ist ein Tagfalter aus der Familie der Edelfalter, gehört zu den großen Schmetterlingsarten und kann bis zu siebeneinhalb Zentimeter Flügelspannweite aufweisen.
- ◆ Für die Überwinterung darf es nicht zu warm sein. Auf den britischen Inseln kann er nicht überwintern, und, durch den menschengemachten Klimawandel, auch in vielen andern Gegenden nicht mehr. Trotzdem mögen die Briten den Trauermantel ganz besonders gerne. In seinem britischen Namen haben sie sogar den ersten Fundort in England verewigt: Camberwell Beauty.
- ◆ Der Trauermantel wird heute in vielen Teilen Europas als mehr oder weniger stark gefährdet eingestuft. Er lebt vor allem in lichten, offenen und feuchten Laubwäldern, man findet ihn aber auch in Obstgärten und an Alleen.

Quellen:

* <https://de.wikipedia.org/wiki/Trauermantel>

* <https://www.plantura.garden/insekten/schmetterlinge/heimische-schmetterlinge>



Extreme gradient boosting - Algorithms

- ◆ **XGBoost** (eXtreme Gradient Boosting)
 - first extreme gradient boosting tree
 - 2016, open-source (Apache License)
- ◆ **LightGBM**
 - 2017, by Microsoft, open source (MIT License)
 - grows the tree “leaf wise”, not “level wise” as XGBoost
- ◆ **CatBoost**
 - 2017, developed by Yandex (“the Google of Russia”), open-source (Apache License)
 - builds symmetric (balanced) trees and has native support for categorical features
- ◆ **HistGradientBoosting**
 - 2019, scikit-learn implementation of extreme gradient boosting
 - inspired by and similar to LightGBM
- ◆ All in active development, no clear “winner”



Extreme Gradient Boosting

- ◆ Can be used for Regression as well as Classification
- ◆ Many differences / additions to "basic" gradient boosting to improve quality and runtime
- ◆ XGBoost
 - Regularization
 - Boosting
 - Handling Missing Values
 - Weighted Quantile Sketch
 - Custom loss function
 - Parallel Processing



XGBoost

1. Regression
2. Classification
3. Mathematical Background
4. Optimizations
5. Summary



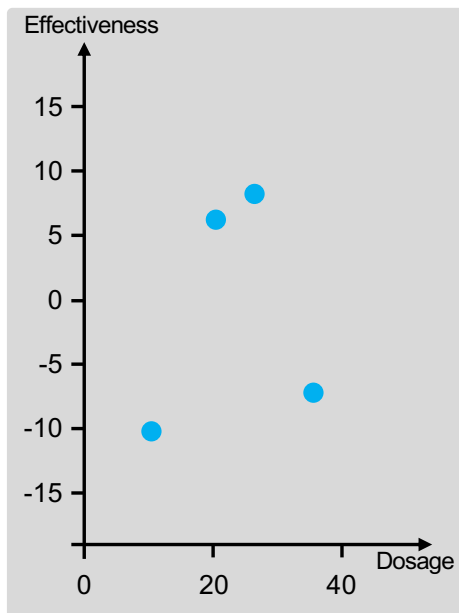
XGBoost vs. “regular” gradient Boosting for Regression

- ◆ Recall “regular” gradient Boosting:
 - Compute “initial prediction” as first model
 - Repeat until max iterations reached or difference of new model very small
 - Build regression tree to adjust for the residuals
 - Combine existing model with the new tree multiplied by the learning rate to update the model
- ◆ XGBoost: same basic approach with the following differences
 - Simplified initial prediction: constant 0.5
 - Each iteration step builds a much more complex “unique” tree instead of a “regular” regression tree
then does the same as regular gradient boosting, except trees are slightly different
- ◆ Note: in this lecture, we discuss the default/most common ways, XGBoost is used – many aspects are parameterized and can be changed, though



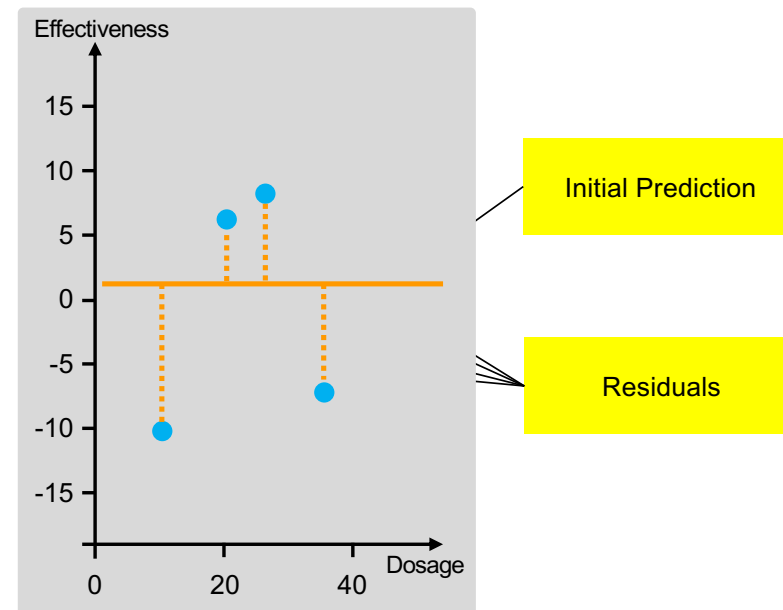
Sample Data Set and Initialization

Dosage	Effectiveness
10	-10
20	7
25	8
35	-7



- Initialization with the constant 0.5 (regardless of the data set): **initial prediction=0.5**

0.5





Iteration: build XGBoost Regression Tree (1)

- ♦ Put all residuals in the leaf of a new tree:

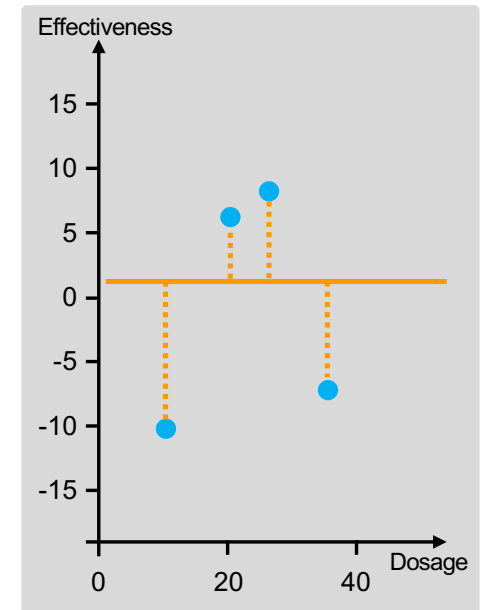
-10.5, 6.5, 7.5, -7.5

λ = regularization parameter, set to 0 here, will discuss later!

- ♦ Compute: $Similarity(root) = \frac{(\sum residuals)^2}{|residuals| + \lambda} = \frac{(-10.5 + 6.5 + 7.5 - 7.5)^2}{4 + \lambda} = 4$
sums up residuals, vastly different residuals cancel each other out

- ♦ Split the tree

- Try binary splits points: test average of each consecutive pair of dosages
- Compute *Similarity* for both resulting leaves
- Compute
 $Gain = Similarity(left\ leaf) + Similarity(right\ leaf) - Similarity(root)$
- Pick the splitpoint with the highest *Gain* and split the leaf recursively (greedy algorithm)
- Very similar to regular decision tree construction, only with a new measure of impurity





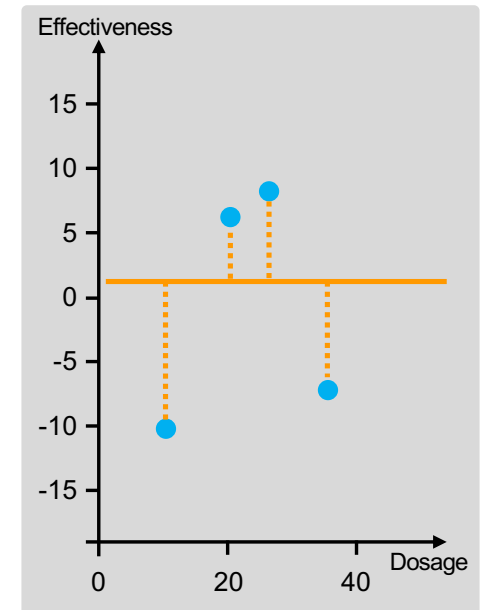
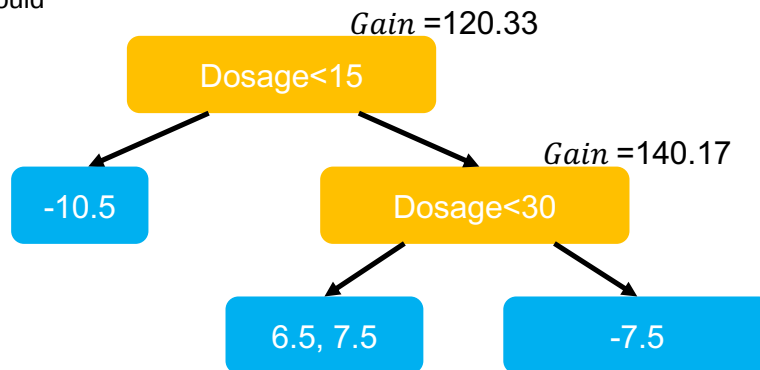
Iteration: build XGBoost Regression Tree (2)

◆ Resulting tree

- Depth limited to 2 levels for the example (parameter *max_depth*, default: 6)

want high bias, low variance
so even if we had 1mio data points, *max_depth* should stay around default

split point should
be mean of
values, so
 $(10+20)/2$

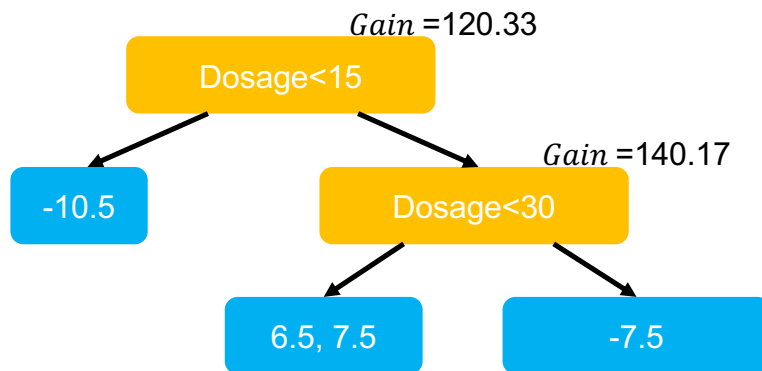




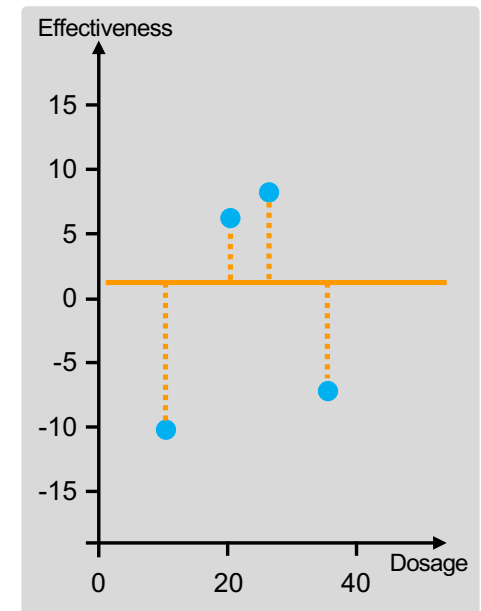
Regularization via γ -Pruning

(hyperparam that should be tuned)

- ◆ Prune all lowest-level-branched with a *Gain* below the threshold γ
- ◆ Here we use $\gamma = 130$, so we do not prune anything
 - Note: if we used a $\gamma > 140.17$, we would prune the whole tree!



- ◆ Note:
 - the *Gain* may be negative, so setting $\gamma = 0$ **does not** turn off pruning!
would have to be - infinity to truly turn it off





L2 Regularization via λ and L1 Regularization

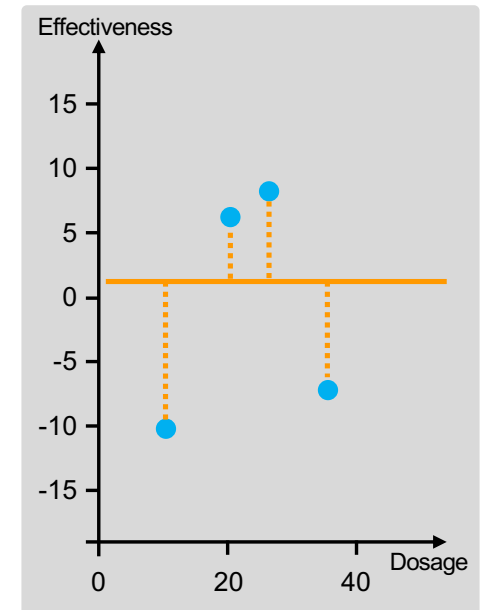
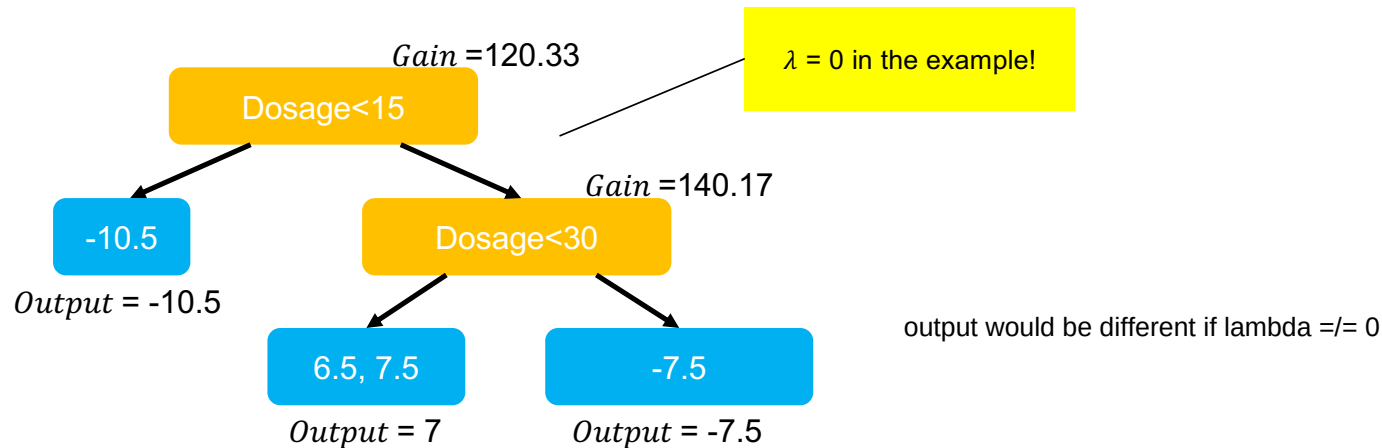
$$\text{Similarity} = \frac{(\sum \text{residuals})^2}{|\text{residuals}| + \lambda}$$

- ◆ **Regularization-Parameter λ :** intended to reduce the sensitivity to individual data points
 - The larger λ , the smaller the *Similarity*
- ◆ Amount of decrease is inversely proportional to the number of residuals in the node (i.e., the fewer residuals, the higher the impact of λ)
 - E.g. $\lambda = 1$:
 - Similarity* for a node with 1 residual : reduced by 50%
 - Similarity* for a node with 10 residuals: reduced by approx. 10%
 - Similarity* for a node with 100 residuals: reduced by approx. 1%
- ◆ Notes:
 - higher values for λ lead to more γ -Pruning, as the *Gain* become smaller
 - May have to reduce γ when increasing λ in order to not over-prune
 - λ is basically the same as Ridge Regularization (L2) does not pull it down to 0 but rather makes all values smaller
 - Lasso (L1) Regularization was not included in the original XGBoost paper, but has been added to the implementation (parameter α , default 0). It restricts the numerator (details omitted here).
values (enumerator) > alpha lead to subtraction of alpha
i.e. selection of features



Iteration: Determine the Output values for the leaves

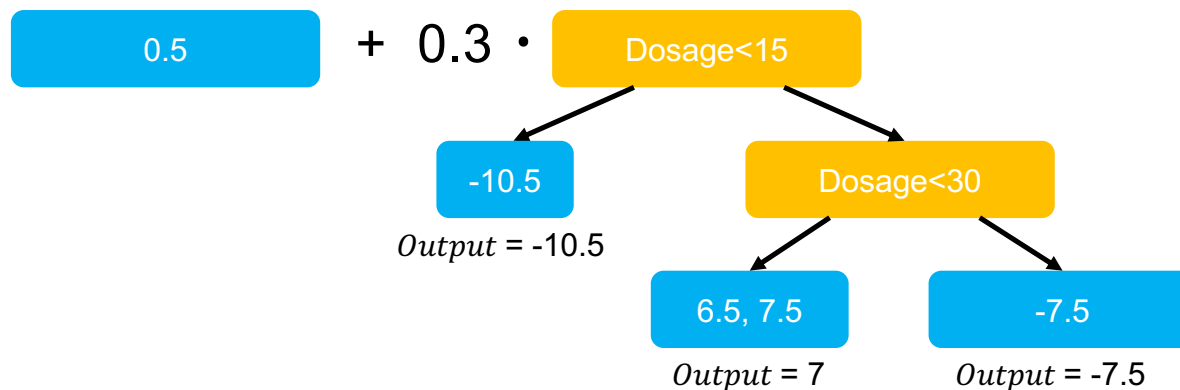
- ◆ $OutputValue(leaf) = \frac{\sum residuals}{|residuals| + \lambda}$
- ◆ Regularization with λ
 - higher values for λ reduce the amount that individual residuals contribute to the *OutputValue*
 - Reduction inversely proportional to the number of residuals in the leaf
 - For $\lambda = 0$, the *OutputValue* is simply the average of the residuals





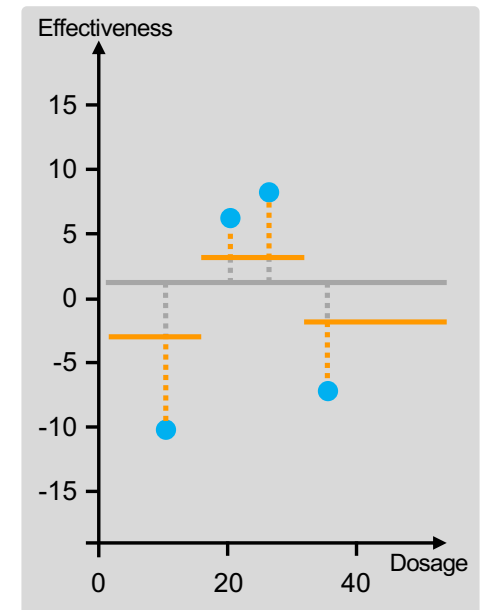
Iteration: combine the models

- Combine the previous model with the new model using the **learning_rate eta (η)** (default 0.3) to control overfitting



Dosage	Effectiveness	Previous Prediction	New Prediction	Previous Residual	New Residual
10	-10	0.5	-2.65	-10.5	-7.35
20	7	0.5	2.6	6.5	4.4
25	8	0.5	2.6	7.5	5.4
35	-7	0.5	-1.75	-7.5	-5.25

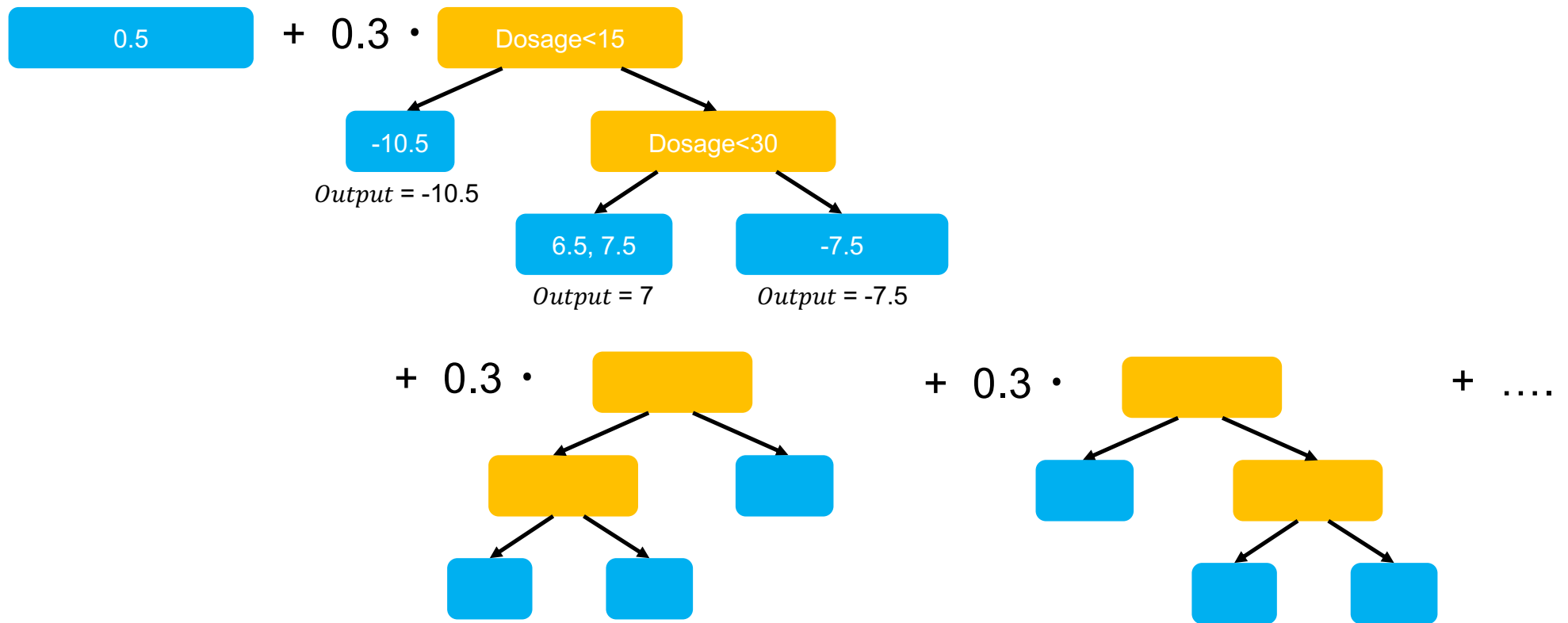
Small steps in the right direction



is not a given that all residuals will move in correct direction but overall sum will be lowered, i.e. improve



Iterate until stopping criterion (max # iterations or stopping criterion) reached





Exercises

Exercise 1

XGBoost Regression Tree Construction





XGBoost

1. Regression
2. Classification
3. Mathematical Background
4. Optimizations
5. Summary



XGBoost for Classification

- ◆ Approach is a hybrid between XGBoost for Regression and regular Gradient Boosting for Classification (GBC)
- ◆ Binary Classification – convert classes to probabilities (as in regular GBC)
 - Example Data Set

Dosage (mg)	Effective (class)
2	no
8	yes
12	yes
18	no



Dosage (mg)	Probability
2	0.0
8	1.0
12	1.0
18	0.0



Similarities and Differences to XGBoost for Regression (1)

- ◆ Identical: start with a constant “predicted probability” (same default: 0.5)
- ◆ Almost identical: Build regression tree on the residuals, but

- $$Similarity = \frac{(\sum residuals)^2}{\sum_i [previousProbability_i \cdot (1 - previousProbability_i)] + \lambda}$$

(instead of $Similarity = \frac{(\sum residuals)^2}{|residuals| + \lambda}$)

- same as in GBC, only with the added λ

- Gain computation: identical

- $$OutputValue = \frac{\sum residuals}{\sum_i [previousProbability_i \cdot (1 - previousProbability_i)] + \lambda}$$

(instead of $OutputValue = \frac{\sum residuals}{|residuals| + \lambda}$)

- same change as for the Similarity calculation
- again, identical to GBC, only with the added λ



Similarities and Differences to XGBoost for Regression (1)

- ◆ Combine the new tree with the existing model – same as for GBC

- Convert previously predicted probability to log(odds)
- Add output value to these weighted by the learning_rate η
- Convert result back to a probability

→ New prediction is $\text{sig} \left(\log \left(\frac{\text{previousProbability}}{1 - \text{previousProbability}} \right) + \eta * \text{OutputValue} \right)$

• this is identical to GBC!

- ◆ Stopping criteria identical



MinChildWeight Regularization

- ◆ XGBoost allows **only nodes in the tree with a $weight > MinChildWeight$** (default: 1)

where the *weight* is the denominator of the *Similarity* minus λ

- i.e., for Regression: $weight = |residuals|$
- i.e., for Classification: $weight = \sum_i [previousProbability_i \cdot (1 - previousProbability_i)]$

- ◆ **Regression**

- *MinChildWeight* acts as a **lower bound on the number of observations in a node**.
- default does not restrict the tree building,
- Increase *MinChildWeight* to enforce a **minimum number of observations per node**

- ◆ **Classification**

- *weight* becomes smaller, the fewer observations are handled by this node and the “purer” the node is (the closer the *previousProbability* are to 0 or 1)
- *MinChildWeight* acts as a **bound on the “purity” of a node**
- default value may restrict the tree construction (may have to decrease)



XGBoost

1. Regression
2. Classification
3. Mathematical Background
4. Optimizations
5. Summary



The Math behind XGBoost (high-level summary)

- Given n training observations (x_i, y_i) and a Loss Function $L(y_i, F(x))$
- XGBoost builds trees by finding output values O_{value} that minimize the equation

$$\left[\sum_{i=1}^n L(y_i, p_i^k) + O_{value} \right] + \gamma T + \frac{1}{2} \lambda O_{value}^2$$

sum of prediction + gamma pruning + regularization

γ -Pruning

Ridge-Regularization

where

p_i^k is the prediction from the previous iteration for training observation i

γ and λ are regularization parameters

T is the number of leaves in the new tree

if 0, then omitted, the bigger the more it influences
lambda controls variance (smaller > high variance)



Regression and Classification – most frequently used Loss functions

- ◆ Most frequently used Loss function for Regression:

$$L(y_i, p_i) = \frac{1}{2} (y_i - p_i)^2 \quad \text{sum of errors squared}$$

- ◆ Most frequently used Loss function for Classification:

$$L(y_i, p_i) = -[y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

- ◆ How to minimize $[\sum_{i=1}^n L(y_i, p_i^k) + O_{value}] + \gamma T + \frac{1}{2} \lambda O_{value}^2$?

- Approximate using **Taylor Expansion with 2 terms** (first and second derivate) same taylor development (?) but cut off at a certain point
- Derive and set to 0 to solve for the optimal O_{value}
- Plug in the respective loss functions given above
- ➔ Result: the *Similarity* and *OutputValue* equations shown in the previous parts of this lecture!

- ◆ XGBoost implementation provides many more Loss functions to choose from

- e.g. for multi-class classification



XGBoost

1. Regression
2. Classification
3. Mathematical Background
4. Optimizations
5. Summary



Approximate Greedy Algorithm for finding the split-points of the regression trees

How to find a split-point with the maximum gain during tree-construction?

- ♦ **Exact algorithm** look at **every** possible split point for every **training** feature
 - very slow for many training examples and/or many features
 - parameter *tree_method: exact*
- ♦ **Approximate Greedy Algorithm** divides the values into quantiles and only considers the quantile borders
 - much faster, but less exact
 - parameter *tree_method: approx* (default: uses about 33 quantiles) or *tree_method: hist* (similar, slightly different implementation)
- ♦ Note: there are a couple more (and similar) *tree_method* algorithms in the XGBoost implementation, but these are the most commonly used ones



Parallel Learning and Weighted Quantil Sketches

- ◆ For large data sets, XGBoost splits the data into subsets and **parallelizes the computation of the tree construction**
 - Multithreading on multiple Cores / CPUs / GPUs
- ◆ To speed this up further, **Sketches** are used to compute the **Quantiles**
 - A **sketch** is a small data structure that lets you **calculate or approximate certain characteristics** of the original data.
 - Extremely simple example: random sample of 1000 values.
 - median of the sample = **roughly** median of the data.
 - mean of the sample = **approx.** mean of the data
 - distribution of the sample = **approx.** the same as the distribution of the data
 - Generally, more elaborate structures are used (e.g. Bloom filters)
 - hashes values to x bits
 - to check if value is not in data set, hash value and check if bit is 0
 - > will definitely answer if value is not part of data set
 - but is not definite when bit = 1
 - > is approximate
- ◆ **Weighted Quantiles** → focus on the hardest to learn parts of the data set



Video

approximate greedy algorithm:

uses quantiles (i.e. percentile split of data set into groups of same size) instead of only values as split points

split can be made via multiple quantiles

the more quantiles, the longer the algorithm will take (AGA uses about 33 as default)

33 quantiles would result in 34 groups of same sample size

note: standard implementation runs on multiple cores instead of computers (cf. video | sketch algorithm)

sketch: creates histogram with which to make approx quantiles

weighted quantiles: each observation

wt and quantiles are determined by sum of weights instead of num

of observations

the higher the confidence in the prediction (e.g. 0.09), the lower the weight and inverse

Approx. Greedy Algorithm

Parallel Learning

Weighted Quantil Sketches

<https://www.youtube.com/watch?v=oRrKeUCEbq8>

(from 1:33 to 16:13)

weighted quantiles are split, so that their weights are similar > as such, high confidence groups are summarized into one group whereas low confidence values are split apart

sparsity aware split (ie. missing values): split table into two > those with values and those with missing ones

calc gain for splits same as before but for each gain(left)/ gain(right), the missing values are added to each set

other way of dealing with missing values: either leave them out or imputation (?) (assign mean of all values as default)

or predict missing value with additional model



sketch: each core gets subset of data set and returns histogram for only that subset which are then combined



Exercises

Exercise 2

Optimizations





Handling Missing Value with Sparsity-Aware Split Finding

- ◆ Real data: usually many **missing values** (“data is sparse”)
 - Reasons: bad data quality and/or feature engineering/encoding/augmentation
- ◆ How to handle?
 - **Option 1**: explicit “missing value handling” (→ not discussed here, see Data Science class)
 - **Option 2**: use an algorithm that can handle missing values
- ◆ XGBoost can handle missing values:
 - Default direction is assigned to each tree using the missing values in the training data, such that the training loss function is minimized
 - missing values are assigned to the default direction when using the model for new data



Further Performance Optimizations

◆ Cache-Aware Access

- XGBoost is aware of the memory hierarchy (CPU-Cache → RAM → SSD/HDD) and optimizes computation by keeping the most frequently used values in the CPU-Cache

◆ Blocks for Out-of-Core Computations for data sets too large to fit into RAM

- If data set is too large to fit into RAM, it is compressed while using the SSD/HDD
 - Trading CPU for SSD/HDD performance
- If multiple SSD/HDDs are available, Sharding is used

◆ Random Sampling

- Of Observations: build each tree for a (different) random subset of the data
- Of Features: build each tree for a (different) random subset of the feature
→ similar to Random Forests



XGBoost

1. Regression
2. Classification
3. Mathematical Background
4. Optimizations
5. Summary



Key Takeaways for XGBoost



- ◆ Basis of XGBoost: **Gradient Boosting**
- ◆ **XGBoost**
 - Tree based ensemble machine learning algorithm
 - Performance due to accurate approximation algorithms.
 - Higher predicting power & performance than “basic” gradient boosting
 - Popular and frequently winner of competitions in data science / ML
- ◆ Improvements: **highly regularized** to control/avoid overfitting
- ◆ Missing Values: **Sparsity Aware Split Finding**
- ◆ Performance
 - **Approximate Greedy Algorithm**
 - **Weighted Quantile Sketch & Parallel Learning**
 - **Cache – Aware Access & Blocks for Out-of-core Computation**



References

- ◆ Original XGBoost paper: <https://dl.acm.org/doi/10.1145/2939672.2939785>
- ◆ XGBoost Documentation: <https://xgboost.readthedocs.io/en/stable/>
- ◆ StatQuest Videos on XGBoost:
 - <https://www.youtube.com/watch?v=OtD8wVaFm6E>
 - <https://www.youtube.com/watch?v=8b1JEDvenQU>
 - <https://www.youtube.com/watch?v=ZVFeW798-2I>
 - <https://www.youtube.com/watch?v=oRrKeUCEbq8>